

COS 125

File IO and Etc.

Objectives:

Utilizing all of the concepts we've covered in the course so far; we can apply data processing to information stored in external files. Reading and writing to external files gives greater options for processing content and storing the results of the program.

Working with Files

- Files must be "opened" and 'closed.'
- Files can be read, data stored in variables, and processed.
- Program results can be written to external files for permanent storage.

File Management: Submit all lab files inside a compressed folder for this lab. DO NOT include other work, only this lab's specific files. Points will be deducted for including non-lab files.

Reading data into the program from a file does create a need to process that data into appropriate data types. Is the data set up as a block of text that is best used in dictionaries, lists, tuples, as digit, etc.

Lab: You may work alone or in groups. If you work in groups, all members of the group should submit work individually and put a collaboration note that you worked together. Do NOT use AI or other resources outside of class concepts.

For practice, run all the example code, even if you know what it will do. Practice helps with learning.

Instructions:

- Open VSCode or your preferred IDE.
- Create a new folder called Lab10.

Demonstrate your working code to the TA or MLA(s) before leaving lab sessions early. Work consistently throughout lab, ask questions, and submit what you have completed at the end of lab.

Submit what you have completed before leaving lab. If you want to complete more outside of lab, that's great for your growth, but submit what you have done before leaving.

Part 1: Exploring Examples (Required)

Create a python file `yourname_exploringLab10.py`

Run the code below for some practice with loops and lists and review the results – ask questions if you don't understand why something did what it did:

Example 1: Opening a file

Be sure the `names.txt` file is in the same folder as the `.py` file you're using. Run the code below.

```
namesFile= open('names.txt','r')
print(namesFile)

for name in namesFile:
    print(name)
```

- What does the "r" do?
- Why does the `print(namesFile)` not print the names?
- What makes the blank space in the print out from the for loop?
- Close the file properly.

```
Parker River Rowan Avery Riley Logan Quinn
Jordan Cameron Angel Carter Emery Kai Nova
```

Example 2: Mode change for external file

Using the appropriate file mode flag (`r`, `w`, `a`, `x`) write code to open a file the `names.txt` file so you can add more names to the end of the file (append to the file).

- What is the difference? Can you close the file and open it again in the same `.py` file?
- Why would you want to have a file opened as read only or write only?

Example 3: Setting up data to write to an external file

Continuing with the code above, using the `names.txt` file opened in append mode. Create a loop that allows the user to enter as many names as they would like and append them to a **list** of names variable named `newNames`. **Do not write the names to the file yet.**

- Does the loop need to access the file?

- Will any changes be made to the file while processing data in the code?

Example 4: Append (write) to the external file

With the file still open from Example 3, let's write the new names to the names.txt file. The file write() function uses string data type only. You may have to convert to str().

```
namesFile.write(str(newNames))
```

- Now close the file.
 - Check if the new names are appended to the file.
-

Part 2: Exercises (Optional)

Exercise 1: Paycheck calculator file

Create a new python file named **last_name_lab10_exercise1.py**

- Write python code that allows users to enter their name, employee id, and hourly pay rate. Calculate what their pay is for a 40 hour work week. Using 23% as the withholding rate calculate the total withholdings. Calculate the net pay for the week (total – withholding).
- Using the correct external file editing mode(s), set the file open() function to create a new text file that can be written to.
- Set up the program so the user can enter another employee's data until the user indicates they no longer want to repeat.
- Output to save to the file should be: Employee Name, ID, Hourly Rate, Total pay, Withholding, Net Total pay for the week. Convert data to appropriate data type to write() to the external file. Write all employees and their information to the external file. The \n formatting character works for external files.
- Check the external file has all of the information you entered.
- Include the external files for this lab in the submissions.

ASK QUESTIONS if you don't understand why something works the way it does, if you want help brainstorming how to approach the concepts, or other questions.

Exercise 2: Create and write numbers in file

Create a new python file named **last_name_lab10_exercise2.py**

- Create a new external file named numbers.txt in write mode
- Generate 10 random numbers and write them to the external numbers.txt file.

- In the python code, close the file.

Exercise 3: Read and Process numbers from File

Create a new python file named **last_name_lab10_exercise3.py**

- Open the external file named numbers.txt in read mode
- Read in the data from the file
- Print the content of the file while checking the type() of the data
- Based on the data type read in, determine how you can split the numbers from the file apart into individual numbers
- Convert string numbers read to numeric values
- Calculate the sum of all of the numbers from the file.
- Perform appropriate file settings to change the mode of the file at this point in the python code to allow you to write the total of the numbers into the external file.
- In the python code, close the file.

Exercise 4: Find a number from an external file

Create a new python file named **last_name_lab10_exercise4.py**

- Open the external file named numbers.txt, that you created before, in **read** mode
- Read in the data from the file in a manner that converts to a list of numbers without hidden characters (e.g. \n \t ' ')
- Print the contents of the file
- Search the data imported to see if the number 9 is in the file.
- Print "found it" if it is, 'nope not present' if not.
- Be sure to modify file mode settings as appropriate.

```
['1', '7', '16', '12', '1', '4', '9', '13', '19', '19']  
Nope not here  
Nope not here  
Nope not here  
Nope not here  
Nope not here  
Nope not here  
Found it  
Nope not here  
Nope not here  
Nope not here
```

Exercise 5: All in One open/close

Create a new python file named `last_name_lab10_exercise5.py`

- Open the external file named `numbers.txt`, that you created before, in read mode (using `open()` as file:).

```
1 with open("numbers.txt", "r") as numsFile:  
2     <python code>  
3 #when| outdented the file closes.  
4
```

- Read in the data from the file in a manner that converts to a list of numbers without hidden characters (e.g. `\n`, `\t`, `' '`). This will involve reading the lines, stripping the newline characters, and converting each string to an integer.
- Calculate statistical assessment of the values:
 - I. the average number of values in the file
 - II. count of values in the file
 - III. min value
 - IV. max value
 - V. total of the values in the file
- Create a statistical report of all numbers in the file, use the string `"NaN"` if not found.
- Using `'with open()'` set up a new external file to write the nicely formatted statistical report to.

Submit all lab files inside a compressed folder for this lab. DO NOT include other work, only this lab's specific files.

When all challenges are done, or the lab time ends submit to BrightSpace. If leaving early let the instructor know before you leave. Submit whatever you have completed before leaving. If you wish to work on the labs to further your own understanding after the lab ends, this is encouraged, but be sure to submit what you did during lab.

When done, submit to BrightSpace and let the instructor know before you leave.